MARK HANEY

STAFF SOFTWARE ENGINEER | RUST BACKEND ENGINEER | DISTRIBUTED SYSTEMS | CLOUD-NATIVE PLATFORMS

Dardanelle, AR
https:// www.linkedin.com/in/markhaney-33b675ba
+1 (430) 279-2231| markhaney.dev@outlook.com

PROFESSIONAL SUMMARY

Staff Software Engineer specializing in **Rust 1.60–1.78**, **Tokio 1.x**, **Axum 0.6–0.7**, and cloud-native backend engineering for SaaS, fintech, public-sector, and infrastructure platforms, with deep experience modernizing legacy services into high-performance microservices, designing safe concurrent systems, improving API latency and reliability, reducing production incidents through observability and automated testing, and leading teams across architecture, CI/CD, Kubernetes, PostgreSQL, MongoDB, Redis, and AWS/GCP while delivering secure, maintainable systems that support product growth, platform scale, business value, and long-term engineering quality.

TECH SKILLS

- **Languages: Rust 1.60–1.78**, TypeScript 4.x–5.x, JavaScript ES6+, Python 3.8–3.12, SQL, Bash
- **Rust Backend: Tokio 1.x**, **Axum 0.6–0.7**, Actix Web 3.x–4.x, Serde 1.x, SQLx 0.6–0.7, Diesel 1.4–2.x, Reqwest 0.11, Clap 3.x–4.x, Rayon 1.x, Tracing 0.1
- **Architecture: Microservices**, distributed systems, event-driven architecture, REST APIs, gRPC with Tonic 0.8–0.11, domain-driven design, system design, API versioning
- **Databases:** PostgreSQL 12–16, MongoDB 4.x–7.x, Redis 6.x–7.x, MySQL, query optimization, indexing, migrations
- **Cloud & DevOps:** AWS, GCP, Docker 20.x–25.x, Kubernetes 1.22–1.29, Terraform 1.x, GitHub Actions, GitLab CI, Jenkins, CI/CD, Linux
- **Testing & Quality:** Unit testing, integration testing, property-based testing with Proptest, Criterion.rs benchmarking, cargo-nextest, contract testing, code review
- **Observability:** OpenTelemetry, Prometheus, Grafana, CloudWatch, structured logging, distributed tracing, alerting, incident response
- **Frontend:** React 17–18, TypeScript, Redux Toolkit, HTML5, CSS3, API integration, dashboard development

PROFESSIONAL EXPERIENCE

Staff Software Engineer

Relatio

Jun 2022 – Apr 2026
Las Vegas, NV

- Led the design of **Rust 1.60–1.78** backend services using **Tokio 1.x**, **Axum 0.6–0.7**, PostgreSQL, and Redis to support high-volume SaaS workflows with safer concurrency and predictable runtime behavior.
- Modernized legacy Node.js and Python services into **Rust microservices**, reducing average API latency by **28%** through async request handling, SQL query tuning, and better connection pooling.
- Resolved recurring production timeout issues by replacing blocking I/O paths with **Tokio async tasks**, structured retries, and timeout boundaries that improved service reliability by **22%**.
- Implemented a new event-processing feature with **Kafka**, Rust consumers, idempotent handlers, and dead-letter queues to prevent duplicate processing during retry storms and partial failures.

- Built shared Rust crates for authentication, error handling, tracing, and request validation, giving multiple teams reusable patterns and reducing duplicated backend code by **18%**.
- Migrated service observability from scattered logs to **OpenTelemetry**, Prometheus, and Grafana dashboards, making slow queries, memory pressure, and queue lag easier to diagnose.
- Fixed a memory growth issue caused by unbounded task spawning by introducing bounded channels, backpressure, and cancellation-safe workers in **Tokio 1.x** services.
- Improved CI/CD quality gates with **cargo clippy**, cargo-nextest, Docker validation, and GitHub Actions, reducing failed deployment rollbacks by **20%**.
- Designed secure API authorization flows using JWT, role-based access control, and service-to-service authentication to protect sensitive platform operations.
- Reviewed architecture proposals, mentored engineers on **Rust ownership**, lifetimes, async patterns, and error modeling, and helped teams avoid unsafe shortcuts in production code.
- Partnered with product, QA, DevOps, and support teams to debug customer-impacting issues quickly while translating business requirements into practical backend designs.
- Introduced performance benchmarks with **Criterion.rs**, load testing, and regression thresholds so critical APIs could be optimized before reaching production traffic.

Senior Software Engineer

AppFolio

Feb 2018 – May 2022
Goleta, CA

- Developed backend services for property-management and financial workflows using **Rust 1.39–1.60**, Actix Web 3.x–4.x, PostgreSQL, Redis, and TypeScript-based internal tools.
- Rebuilt selected high-throughput API components from Ruby and Node.js into **Rust services**, improving request throughput by **24%** without increasing infrastructure cost.
- Solved intermittent payment reconciliation errors by adding idempotency keys, transaction boundaries, and stronger validation around distributed job processing.
- Migrated older REST endpoints into versioned APIs with consistent schema validation, structured error responses, and backward-compatible contracts for frontend consumers.
- Implemented background workers in **Rust** for document processing, notification dispatch, and scheduled reconciliation jobs with reliable retries and failure isolation.
- Reduced database lock contention by **19%** through PostgreSQL index tuning, query-plan review, batch-size adjustments, and improved transaction scoping.
- Created integration tests around billing, ledger updates, and customer account states to catch edge cases before they reached production releases.
- Improved developer workflow by adding reusable Docker Compose environments, seeded test data, and CI checks that made local debugging more consistent.
- Collaborated with React engineers to expose faster backend APIs for dashboards, filters, and reporting pages while keeping API contracts clear and stable.
- Mentored mid-level engineers on **Rust error handling**, clean service boundaries, testing discipline, and practical migration planning from legacy systems.

Software Engineer

Blackbaud

Jul 2016 – Jan 2018
Charleston, SC

- Built nonprofit and fundraising platform services using **Rust 1.16–1.23** for performance-sensitive tools, along with C#, JavaScript, SQL Server, PostgreSQL, and REST APIs.
- Created internal Rust command-line utilities with **Clap**, Serde, and structured logging to validate data exports and reduce manual investigation time by **21%**.
- Fixed recurring import failures caused by malformed donor records by adding schema checks, clearer error messages, and retry-safe validation before batch processing.
- Improved batch-processing reliability by **27%** through better chunking, transaction handling, and background job monitoring across large customer datasets.
- Helped migrate legacy API modules into cleaner service layers with stronger unit tests, reducing regression risk during fundraising feature releases.
- Implemented reporting improvements for customer-facing dashboards by optimizing SQL queries, caching expensive lookups, and simplifying backend aggregation logic.
- Resolved production bugs involving duplicate records, stale cache values, and inconsistent status updates by tracing the full request path across services.
- Worked closely with QA and product teams to convert vague customer issues into reproducible cases, targeted fixes, and safer release notes.
- Supported CI improvements with automated test execution, code review standards, and deployment verification steps for backend and integration changes.

Software Developer

Granicus

May 2014 – Jun 2016
Dallas, TX

- Developed public-sector web platform features using JavaScript, C#, SQL Server, REST APIs, and early service-oriented patterns for government communication systems.
- Improved request handling for citizen-facing workflows by **18%** through API cleanup, query tuning, and removal of unnecessary synchronous processing.
- Fixed notification delivery bugs where failed retries caused missing status updates by adding better retry tracking, logging, and operational visibility.
- Implemented new administrative features for content review, publishing approval, and audit history so government users could manage workflows more safely.
- Migrated older modules into cleaner layered services with stronger validation and more maintainable controller logic for long-term platform stability.
- Reduced manual support investigation time by **25%** by improving error messages, adding searchable logs, and documenting common failure patterns.
- Collaborated with senior engineers to review production defects, prepare hotfixes, and validate releases without disrupting active customer portals.
- Used flexible backend, database, and frontend skills to support urgent customer issues while still improving code quality during regular feature delivery.

Junior Software Developer

Vology

Jun 2013 – Apr 2014
Gainesville, FL

- Supported infrastructure and business application development using JavaScript, PHP, Python scripts, MySQL, Linux, and internal automation tools for operational teams.
- Reduced repetitive ticket triage work by **17%** by building small automation scripts that parsed logs, organized support data, and surfaced common failure patterns.
- Fixed data-entry and reporting issues by improving validation rules, correcting SQL queries, and coordinating with support teams to verify real customer scenarios.
- Helped maintain internal dashboards and backend utilities while learning production debugging, source control, release discipline, and cross-team communication.
- Reduced deployment mistakes by **14%** by documenting environment setup steps, improving checklist quality, and validating changes before handoff to senior engineers.

EDUCATION

Arkansas State University
Bachelor's degree, Computer Science (Sep 2009 – May 2013)